

API 레퍼런스: KIT H10 제어 & Data Interface

XM10의 핵심 가치중 하나는 **KIT H10** 로봇을 직접 설계한 알고리즘으로 제어하는 것입니다. 본 API는 KIT H10과의 연결 상태를 확인하고, 로봇의 현재 상태 데이터를 실시간으로 수신하며, **PIF-Vectors**, **Aux inputs**와 같은 제어 명령을 전송하여 로봇의 움직임을 제어하는 데 필요한 기능을 제공합니다. **xm_api_data.h**에 정의된 **로봇 데이터 및 제어 API**에 대한 상세 레퍼런스입니다. XM10 펌웨어는 사용자가 복잡한 통신 프로토콜(CAN-FD, UART)을 신경 쓰지 않고, 직관적인 전역 객체(**XM**)를 통해 로봇의 상태를 읽고 명령을 내릴 수 있도록 **파사드 (Facade) 패턴**을 제공합니다.

동작 원리 (Operating Principle)

XM10의 제어 시스템은 엄격한 **IPO (Input-Process-Output)** 모델을 따르며, 이는 시스템 내부의 **core_process** 엔진에 의해 2ms(500Hz) 주기로 정확하게 수행됩니다.

The IPO Cycle (2ms Loop)

1. Input (Data Gathering):

- 루프가 시작되면 시스템은 **H10**, **GRF Module**, **IMU Module** 등 연결된 모든 하드웨어로부터 최신 데이터를 수집합니다.
- 수집된 데이터는 물리적 단위(Degree, Nm 등)로 변환되어 **XM.status** 구조체에 업데이트됩니다.
- 사용자는 이 단계에서 항상 **가장 최신의 데이터 스냅샷**을 보장받습니다.

2. Process (User Loop):

- 사용자가 작성한 **User_Loop()**(또는 TSM Loop)가 실행됩니다.
- 사용자는 **XM.status**를 읽어 현재 상태를 판단하고, 제어 알고리즘을 수행합니다.
- 계산된 제어 명령(토크 등)은 **XM_Set...** 함수를 통해 **XM.command** 구조체에 기록(Staging)됩니다.

3. Output (Command Flushing):

- 사용자 루프가 끝나면, 시스템은 **XM.command**에 변경된 사항이 있는지 확인합니다.
- 제어 모드(**XM_CTRL_TORQUE**)인 경우, 변경된 명령을 실제 하드웨어(CAN Bus)로 전송합니다.

4. Data Logging(MSC) or Streaming(CDC):

- Input Data, Process, Output Data가 처리된 후 Data Logging or Data Streaming을 수행합니다.
- USB Memory가 연결된 경우 사용자 정의 데이터를 2ms 마다 Memory에 저장합니다.
- PC와 USB로 연결되어 시리얼 포트로 **AGRB MON START** 문자열을 XM10으로 전송하면 사용자 정의 데이터를 2ms마다 터미널로 전달합니다. **AGRB MON STOP**을 입력하면 전송을 중단합니다.

Note: 사용자는 데이터를 수신(Receive)하거나 전송(Flush)하는 함수를 직접 호출할 필요가 없습니다. 오직 데이터를 읽고(Read), 설정(Set)하기만 하면 됩니다. **Note:** 데이터를 저장시에 데이터 저장을 위한 복잡한 로직을 수행할 필요가 없습니다. 저장할 데이터 구조체 정의 및 데이터 전송 API 함수를 호출하기만 하면 됩니다.

데이터 구조 (Data Structures)

모든 데이터는 **XmRobot_t** 타입의 전역 인스턴스인 **xm**을 통해 접근합니다.

XmControlMode_t

제어 모드를 정의합니다.

```
typedef enum {
    XM_CTRL_MONITOR = 0, // 제어 명령 전송 안 함 (Safety)
    XM_CTRL_TORQUE  = 1  // 제어 명령 전송 함 (Active)
} XmControlMode_t;
```

XmH10Mode_t

H10 슈트의 현재 동작 상태입니다.

```
typedef enum {
    XM_H10_MODE_STANDBY = 0, // 대기 중
    XM_H10_MODE_ASSIST  = 1, // 보조력 출력 중
} XmH10Mode_t;
```

PVector_t

H10 슈트의 현재 동작 상태입니다.

```
typedef struct {
    int16_t yd; // Desired Position (unit: deg, scaled by 100)
    uint16_t L; // Trajectory Duration (ms)
    uint8_t s0; // Acceleration Profile (deg/s^2)
    uint8_t sd; // Deceleration Profile (deg/s^2)
} PVector_t;
```

IVector_t

H10 슈트의 현재 동작 상태입니다.

```
typedef struct {
    uint8_t epsilon; // Half width of Corridor (deg, scaled by 10)
    uint8_t kp;      // Virtual Spring Magnitude (%)
    uint8_t kd;      // Virtual Damper Magnitude (%)
    uint8_t lambda;  // Impedance Ratio (scaled by 100)
    uint16_t duration; // Transition Duration (ms)
} IVector_t;
```

FVector_t

H10의 현재 동작 상태입니다.

```
typedef struct {
    uint16_t modeIdx;    // Torque Profile Index, Tp : 0.1 ~ 10
    int16_t  tauMax;     // Max Torque (A, scaled by 100)
    uint16_t delay;      // Initial Delay (ms)
    uint16_t zero;       // Dummy data for reset
} FVector_t;
```

XmH10Data_t

H10 웨어러블 로봇 본체에서 수신된 핵심 데이터입니다. 접근 경로는 `XM.status.h10`입니다. 실제 수신받는 데이터는 수정될 수 있습니다.

```
typedef struct {
    bool is_connected;    // 연결 상태

    // --- Info & State ---
    uint32_t h10AssistModeLoopCnt; // H10 보조 모드 루프 카운트 (Assist Mode시작시
count)
    XmH10Mode_t h10Mode;    // H10 동작 모드 (Assist(1)<->Standby(0))
    uint8_t h10AssistLevel; // H10 보조 레벨 (0~10)
    bool isPVectorRHDone;   // RH Pvector Complete Flag
    bool isPVectorLHDone;   // LH Pvector Complete Flag

    // --- Kinematics Data (운동학 정보) ---
    float leftHipAngle;     // 왼쪽 고관절 각도 (Degree)
    float rightHipAngle;    // 오른쪽 고관절 각도
    float leftThighAngle;   // 왼쪽 허벅지 절대각 (Degree)
    float rightThighAngle;  // 오른쪽 허벅지 절대각
    float leftKneeAngle;    // 왼쪽 무릎 각도 (추정치)
    float rightKneeAngle;   // 오른쪽 무릎 각도 (추정치)
    float pelvicAngle;      // 골반 각도 (Tilt)
    float pelvicVelY;       // 골반 각속도

    // --- Gait Data (보행 정보) ---
    bool isLeftFootContact; // 왼쪽 발 착지 여부
    bool isRightFootContact; // 오른쪽 발 착지 여부
    bool gaitState;         // 보행 상태 (boolean)
    uint8_t gaitCycle;      // 보행 주기 (%)
    float forwardVelocity;  // 전방 보행 속도 (m/s)

    // --- Motor Data (모터 상태) ---
    float leftHipTorque;    // 왼쪽 출력 토크 (Nm)
    float rightHipTorque;   // 오른쪽 출력 토크
    float leftHipMotorAngle; // 왼쪽 모터 엔코더 각도 (Degree)
    float rightHipMotorAngle; // 오른쪽 모터 엔코더 각도

    // --- IMU Data (관성 센서 상세 정보) ---
    // Orientation
```

```

float leftHipImuFrontalRoll;    // 왼쪽 고관절 IMU Frontal Roll 각도 (Degree)
float rightHipImuFrontalRoll;
float leftHipImuSagittalPitch; // 왼쪽 고관절 IMU Sagittal Pitch 각도 (Degree)
float rightHipImuSagittalPitch;

// Global Acceleration (m/s^2)
float leftHipImuGlobalAccX; // 왼쪽 고관절 IMU Global 가속도
float leftHipImuGlobalAccY;
float leftHipImuGlobalAccZ;
float rightHipImuGlobalAccX;
float rightHipImuGlobalAccY;
float rightHipImuGlobalAccZ;

// Global Gyroscope (deg/s)
float leftHipImuGlobalGyrX; // 왼쪽 고관절 IMU Global 자이로
float leftHipImuGlobalGyrY;
float leftHipImuGlobalGyrZ;
float rightHipImuGlobalGyrX;
float rightHipImuGlobalGyrY;
float rightHipImuGlobalGyrZ;
} XmH10Data_t;

```

강조 표시한 데이터는 현재 받고 있는 데이터 입니다. 추후 추가되거나 수정될 수 있습니다.

Field Name	Type	Unit	Description
is_connected	bool	-	H10와의 통신 연결 여부 (true: 정상)
h10AssistModeLoopCnt	uint32_t	-	H10 보조 모드 루프 카운트
h10Mode	XmH10Mode_t	-	현재 H10 동작 모드 (STANDBY / ASSIST)
h10AssistLevel	uint8_t	1~9	H10에 설정된 보조 강도 레벨
isPVectorRHDone	bool	-	RH의 P vector가 완료되었음을 알리는 플래스
isPVectorLHDone	bool	-	LH의 P vector가 완료되었음을 알리는 플래스
leftHipAngle	float	deg	왼쪽 고관절 각도 (Extension < 0 < Flexion)
rightHipAngle	float	deg	오른쪽 고관절 각도
leftThighAngle	float	deg	왼쪽 허벅지 절대 각도 (수직 기준)
rightThighAngle	float	deg	오른쪽 허벅지 절대 각도
leftKneeAngle	float	deg	왼쪽 무릎 각도 (추청지)
rightKneeAngle	float	deg	오른쪽 무릎 각도 (추청지)
pelvicAngle	float	deg	골반 좌우 기울기 (Tilt)
pelvicVelY	float	deg/s	골반 회전 각속도
isLeftFootContact	bool	-	왼쪽 발 착지 여부 (true: 지면 접촉)

Field Name	Type	Unit	Description
isRightFootContact	bool	-	오른쪽 발 착지 여부
gaitState	uint8_t	-	보행 중 여부 (0 / 1)
gaitCycle	uint8_t	%	보행 주기 진행률 (0 ~ 100)
forwardVelocity	float	m/s	전방 속도 (추정치)
leftHipTorque	float	Nm	왼쪽 모터 현재 출력 토크 (Feedback)
rightHipTorque	float	Nm	오른쪽 모터 현재 출력 토크 (Feedback)
leftHipMotorAngle	float	deg	왼쪽 모터 현재 엔코더 각도 (Feedback)
rightHipMotorAngle	float	deg	오른쪽 모터 현재 엔코더 각도 (Feedback)
leftHipImuFrontalRoll	float	deg	왼쪽 고관절 IMU Frontal Roll 각도
rightHipImuFrontalRoll	float	deg	오른쪽 고관절 IMU Frontal Roll 각도
leftHipImuSagittalPitch	float	deg	왼쪽 고관절 IMU Sagittal Pitch 각도
rightHipImuSagittalPitch	float	deg	오른쪽 고관절 IMU Sagittal Pitch 각도
leftHipImuGlobalAccX	float	m/s^2	왼쪽 고관절 IMU Global 가속도 X
leftHipImuGlobalAccY	float	m/s^2	왼쪽 고관절 IMU Global 가속도 Y
leftHipImuGlobalAccZ	float	m/s^2	왼쪽 고관절 IMU Global 가속도 Z
rightHipImuGlobalAccX	float	m/s^2	오른쪽 고관절 IMU Global 가속도 X
rightHipImuGlobalAccY	float	m/s^2	오른쪽 고관절 IMU Global 가속도 Y
rightHipImuGlobalAccZ	float	m/s^2	오른쪽 고관절 IMU Global 가속도 Z
leftHipImuGlobalGyrX	float	deg/s	왼쪽 고관절 IMU Global 자이로 X
leftHipImuGlobalGyrY	float	deg/s	왼쪽 고관절 IMU Global 자이로 Y
leftHipImuGlobalGyrZ	float	deg/s	왼쪽 고관절 IMU Global 자이로 Z
rightHipImuGlobalGyrX	float	deg/s	오른쪽 고관절 IMU Global 자이로 X
rightHipImuGlobalGyrY	float	deg/s	오른쪽 고관절 IMU Global 자이로 Y
rightHipImuGlobalGyrZ	float	deg/s	오른쪽 고관절 IMU Global 자이로 Z

XM_GRF_SPACE_e

GRF Module의 왼쪽/오른쪽 연결 id입니다.

```
typedef enum {
    XM_SPACE_LEFT = 1,
    XM_SPACE_RIGHT,
```

```
XM_SPACE_UNKNOWN,
} XM_GRF_SPACE_e;
```

XmGrfData_t

GRF Module의 데이터입니다.

```
typedef struct {
    bool    is_left_grf_connected; // 왼쪽 GRF 모듈 연결 상태
    bool    is_right_grf_connected; // 오른쪽 GRF 모듈 연결 상태

    // --- Left Foot Data ---
    // sensorSpace가 LEFT(1)인 패킷의 데이터
    uint32_t leftLastUpdateTick; // 데이터 수신 시각 (ms)
    XM_GRF_SPACE_e leftSensorSpace; // 1=왼발, 2=오른발
    uint8_t leftRollingIndex; // 0-199 패킷 시퀀스
    uint8_t leftSensorData[XM_GRF_CHANNEL_SIZE]; // 14개 채널 값 (0~255 Raw Value)
    uint8_t leftBatteryLevel; // 배터리 잔량 (0~100)
    uint8_t leftStatusFlags; // 상태 플래그

    // --- Right Foot Data ---
    // sensorSpace가 RIGHT(2)인 패킷의 데이터
    uint32_t rightLastUpdateTick;
    XM_GRF_SPACE_e rightSensorSpace; // 1=왼발, 2=오른발
    uint8_t rightRollingIndex; // 0-199 패킷 시퀀스
    uint8_t rightSensorData[XM_GRF_CHANNEL_SIZE]; // (0~255 Raw Value)
    uint8_t rightBatteryLevel;
    uint8_t rightStatusFlags;
} XmGrfData_t;
```

강조 표시한 데이터는 현재 받고 있는 데이터 입니다.

Field Name	Type	Unit	Description
is_left_grf_connected	bool	-	왼쪽 GRF 모듈 연결 상태
is_right_grf_connected	bool	-	오른쪽 GRF 모듈 연결 상태
leftLastUpdateTick	uint32_t	ms	왼쪽 데이터 수신 시각
leftSensorSpace	XM_GRF_SPACE_e	-	1=왼발, 2=오른발
leftRollingIndex	uint8_t	-	왼쪽 0-199 패킷 시퀀스
leftSensorData[14]	uint8_t	-	왼쪽 14개 채널 값 (0~255 Raw Value)
leftBatteryLevel	uint8_t	-	왼쪽 배터리 잔량 (0~100)
leftStatusFlags	uint8_t	-	왼쪽 상태 플래그
rightLastUpdateTick	uint32_t	ms	오른쪽 데이터 수신 시각

Field Name	Type	Unit	Description
<code>rightSensorSpace</code>	<code>XM_GRF_SPACE_e</code>	-	1=왼발, 2=오른발
<code>rightRollingIndex</code>	<code>uint8_t</code>	-	오른쪽 0-199 패킷 시퀀스
<code>rightSensorData[14]</code>	<code>uint8_t</code>	-	오른쪽 14개 채널 값 (0~255 Raw Value)
<code>rightBatteryLevel</code>	<code>uint8_t</code>	-	오른쪽 배터리 잔량 (0~100))
<code>rightStatusFlags</code>	<code>uint8_t</code>	-	오른쪽 상태 플래그

XmImuData_t

XSENS IMU(mti-630)의 데이터입니다. `XM_EnableExternalImu` 함수를 호출해야 사용할 수 있습니다. (하드웨어 연결 필수) `XM_EnableExternalImu` 함수 호출 시 `XM_EXT_ADC_1(PA0)` -> UART Tx / `XM_EXT_ADC_3(PA1)` -> UART Rx로 변경됩니다.

```
typedef struct {
    bool is_connected; // XSENS IMU 모듈 연결 상태
    uint32_t lastUpdateTick; // 데이터 수신 시각 (ms)

    // --- 1. Orientation (Quaternion) ---
    float q_w, q_x, q_y, q_z;

    // --- 2. Calibrated Acceleration (m/s^2) ---
    float acc_x, acc_y, acc_z;

    // --- 3. Calibrated Gyroscope (deg/s or rad/s) ---
    float gyr_x, gyr_y, gyr_z;
} XmImuData_t;
```

강조 표시한 데이터는 현재 받고 있는 데이터 입니다.

Field Name	Type	Unit	Description
<code>is_connected</code>	<code>bool</code>	-	XSENS IMU 모듈 연결 상태
<code>lastUpdateTick</code>	<code>uint32_t</code>	ms	데이터 수신 시각
<code>q_w</code>	<code>float</code>	-	Orientation (Quaternion) w
<code>q_x</code>	<code>float</code>	-	Orientation (Quaternion) x
<code>q_y</code>	<code>float</code>	-	Orientation (Quaternion) y
<code>q_z</code>	<code>float</code>	-	Orientation (Quaternion) z
<code>acc_x</code>	<code>float</code>	m/s^2	Calibrated Acceleration x
<code>acc_y</code>	<code>float</code>	m/s^2	Calibrated Acceleration y
<code>acc_z</code>	<code>float</code>	m/s^2	Calibrated Acceleration z

Field Name	Type	Unit	Description
gyr_x	float	deg/s	Calibrated Gyroscope x
gyr_y	float	deg/s	Calibrated Gyroscope y
gyr_z	float	deg/s	Calibrated Gyroscope z

XmInput_t

로봇 상태 통합 구조체로서 XM.status를 통해 이 구조체에 접근합니다.

```
typedef struct {
    XmH10Data_t h10;
    XmGrfData_t grf;
    XmImuData_t imu;
} XmInput_t;
```

XmOutput_t

로봇 제어 명령 구조체입니다.

```
typedef struct {
    XmControlMode_t control_mode; // 현재 제어 모드

    float assist_torque_rh;
    float assist_torque_lh;

    /* Dirty Flags (User는 몰라도 됨 - Helper 함수가 관리) */
    struct {
        uint8_t torque_rh_updated : 1;
        uint8_t torque_lh_updated : 1;
    } _dirty_flags;
} XmOutput_t;
```

XmRobot_t (Global Instance XM)

```
typedef struct {
    XmInput_t status; // [Read] 센서 데이터 (H10, GRF, IMU)
    XmOutput_t command; // [Write] 제어 명령 (Helper 함수 사용)
} XmRobot_t;
```

• 주요 필드 접근:

- **XM.status.h10.leftHipAngle**: 왼쪽 고관절 각도 (Degree)
- **XM.status.h10.rightHipTorque**: 오른쪽 현재 토크 (Nm)
- **XM.status.grf.leftSensorData**: 왼쪽 FSR 센서 배열

- `XM.status.imu.acc_z`: IMU 수직 가속도
- ...

함수 (Functions)

알고리즘을 시작하기 전, `XM10`이 `KIT H10`의 제어 모듈(`Control Module`)과 안정적으로 통신하고 있는지 반드시 확인해야 합니다.

`XM_IsCmConnected()`

제어 모듈(`CM`)과의 통신 연결이 활성화(`Operational`) 상태인지 확인합니다. 제어 모듈(`CM`)과 `XM10`간 연결은 내부 `Plug and Play` 백그라운드 태스크에 의해 수행되며, `PDO` 데이터를 수신 받은 첫 시점부터 통신 연결을 활성화 상태로 판단합니다.

Syntax

```
bool XM_IsCmConnected(void);
```

Returns

- `true`: 연결이 정상적으로 활성화된 상태입니다.
- `false`: 연결이 끊겼거나 아직 준비되지 않은 상태입니다.

Example

```
#include "xm_api.h"

void Off_loop(void) {
    // CM과 연결이 확인되면 Standby 상태로 전환합니다.
    if (XM_IsCmConnected()) {
        XM_TSM_TransitionTaskTo(s_mainTaskHandle, XM_STATE_STANDBY);
    }
}
```

`XM_SetControlMode`

로봇의 제어 권한(`Control Authority`) 모드를 설정합니다. 안전을 위해 매우 중요한 함수입니다. 기본값은 모니터링모드로 `H10`에 실시간 제어를 하지 않습니다. 실시간 토크 제어를 위해서는 `XM_SetControlMode`에 `XM_CTRL_TORQUE`를 입력해야 합니다.

Syntax

```
void XM_SetControlMode(XmControlMode_t mode);
```

Parameters

- **mode**: 설정할 모드
 - **XM_CTRL_MONITOR** (0): **모니터링 모드**. 제어 명령을 전송하지 않습니다. (기본값, 안전)
 - **XM_CTRL_TORQUE** (1): **토크 제어 모드**. 설정된 토크 명령을 모터로 전송합니다.

Safety Logic

- 모드가 변경될 때(예: Monitor -> Torque), **내부적으로 모든 토크 명령을 즉시 0.0으로 초기화**합니다. 이는 제어 시작 순간에 급격한 움직임(Jerk)이 발생하는 것을 방지하기 위함입니다.

Example

```
// 알고리즘 시작 시 토크 제어 모드 활성화
void Active_Entry(void) {
    XM_SetControlMode(XM_CTRL_TORQUE);
}

// 알고리즘 종료 시 안전하게 모니터링 모드로 복귀
void Active_Exit(void) {
    XM_SetControlMode(XM_CTRL_MONITOR);
}
```

기존 H10의 보조 모드를 그대로 사용하고자 할 때, 아래의 **XM_SetH10AssistExistingMode**에 true를 입력하여 함수를 호출하여야 합니다.

XM_SetH10AssistExistingMode()

true(1)은 H10 기존 보조 알고리즘 활성화 false(0)은 비활성화. 기본값은 XM10과 H10연결시 기존 보조 알고리즘 비활성화

Syntax

```
void XM_SetH10AssistExistingMode(bool isSet);
```

Returns

- **true**: H10의 기존 보조 알고리즘 활성화
- **false**: H10의 기존 보조 알고리즘 비활성화

Example

```
#include "xm_api.h"

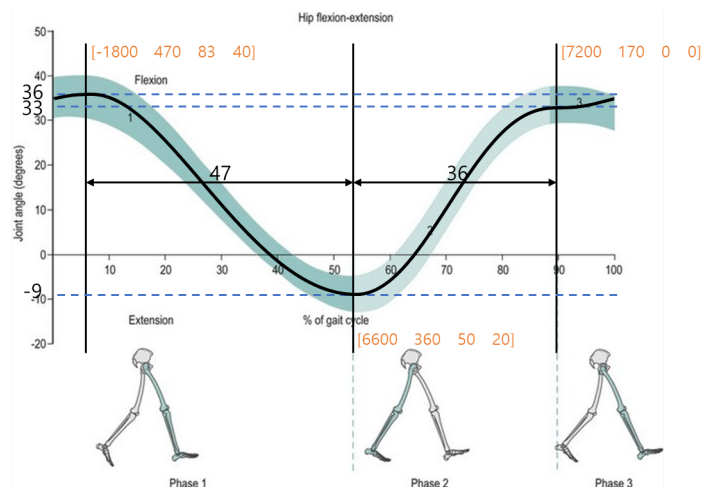
void Off_Entry(void) {
    XM_SetH10AssistExistingMode(true);
}
```

데이터 송신 (Data Outputs, Control inputs)

P-Vector, **I-Vector**, **F-Vector** (PIF-Vectors)와 다양한 제어 명령을 통해 **KIT H10**의 움직임을 정밀하게 설계할 수 있습니다. **PIF-Vector**와 같은 사전정의 된 제어 기법의 자세한 내용에 대해서는 [[angel Robotics-Control Algorithm](#)](작성 예정)에서 확인할 수 있습니다.

XM_SendPVector()

위치 기반 궤적(**P-Vector**)을 전송하여, 지정된 시간 동안 목표 위치까지 부드러운 궤적을 그리며 움직이도록 명령합니다. 반드시, **I-Vector**에 의해 사전에 임피던스 제어 파라미터가 설정되어 있어야 합니다. **P-Vector** 전송시, 모터드라이버에서 5차 **polynomial** 형태의 위치 궤적을 생성합니다.



3번의 명령으로 hip joint trajectory 생성 가능

▲ Figure 1. P-Vector 기반 위치 궤적 생성 예시

Syntax

```
void XM_SendPVector(SystemNodeID_t nodeId, const PVector_t* pVector);
```

Parameters

- nodeId**: 명령을 전달할 관절 (SYS_NODE_ID_RH 또는 SYS_NODE_ID_LH).
- pVector**: 목표 위치(yd), 이동 시간(L), 시작 가속도(s0), 끝 감속도(sd) 정보가 담긴 PVector_t 구조체의 포인터.

Returns 없음.

pVector 구조체 주요 멤버:

멤버	설명	단위	타입
yd	목표 위치	degree, scaled by 100	int16_t
L	이동 시간	ms	uint16_t

멤버	설명	단위	타입
<code>s0</code>	시작 가속도	deg/s ²	uint8_t
<code>sd</code>	끝 감속도	deg/s ²	uint8_t

Example

```
static void UpdatePassiveMode(void)
{
    // XM.status.h10 캐시에서 현재 각도를 읽어옵니다.
    int16_t currentAngleRH = (int16_t)round(XM.status.h10.rightHipMotorAngle *
10.0f);
    int16_t currentAngleLH = (int16_t)round(XM.status.h10.leftHipMotorAngle *
10.0f);

    switch (s_passiveState) {
        case PASSIVE_STATE_SET_IMPEDANCE: {
            // 위치 제어를 위한 임피던스(강성) 설정
            IVector_t stiffImpedance = { .epsilon = 0, .kp = 80, .kd = 1, .lambda
= 0, .duration = 50 };
            XM_SendIVector(SYS_NODE_ID_RH, &stiffImpedance);
            XM_SendIVector(SYS_NODE_ID_LH, &stiffImpedance);
            s_passiveState = PASSIVE_STATE_START_MOTION;
            break;
        }
        case PASSIVE_STATE_START_MOTION: {
            // 첫 목표 각도로 이동하는 P-Vector 전송
            int16_t targetAngle = JOINT_ANGLE_MAX_ANGLE_INT16;

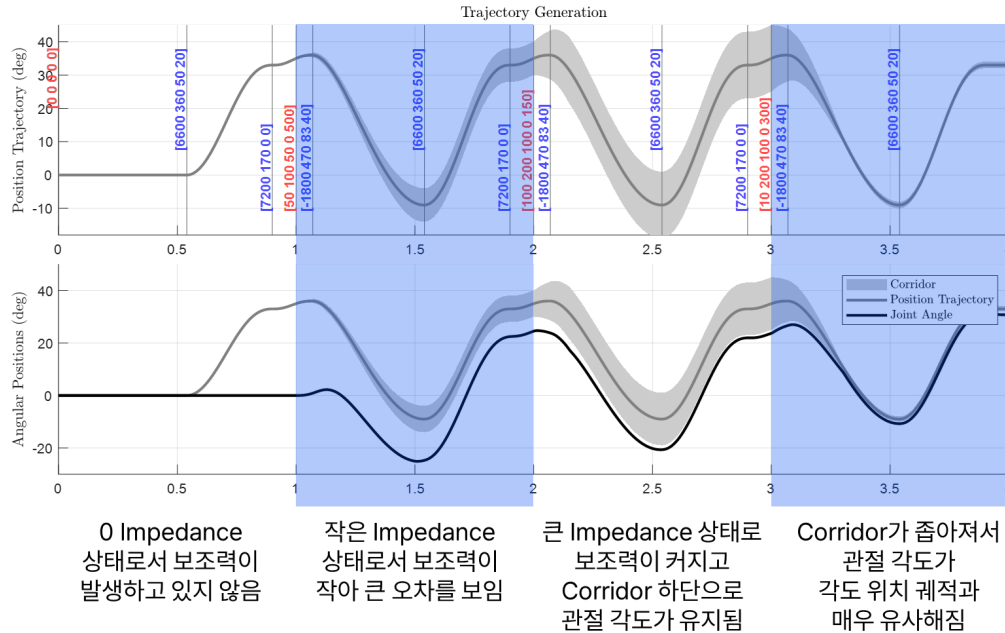
            // 목표 각도로 이동하는 데 필요한 duration 계산
            int16_t angleToMoveRH = abs(targetAngle - currentAngleRH);
            int16_t angleToMoveLH = abs(targetAngle - currentAngleLH);
            uint16_t durationRH = (uint16_t)((((float)angleToMoveRH /
(float)PM_SPEED_RH) * 1000.0f);
            uint16_t durationLH = (uint16_t)((((float)angleToMoveLH /
(float)PM_SPEED_LH) * 1000.0f);

            PVector_t pVecRH = { .yd = targetAngle, .L = durationRH, .s0 =
PM_ACCEL_S0_RH, .sd = PM_ACCEL_SD_RH };
            PVector_t pVecLH = { .yd = targetAngle, .L = durationLH, .s0 =
PM_ACCEL_S0_LH, .sd = PM_ACCEL_SD_LH };
            XM_SendPVector(SYS_NODE_ID_RH, &pVecRH);
            XM_SendPVector(SYS_NODE_ID_LH, &pVecLH);

            s_passiveState = PASSIVE_STATE_MOVING_TO_MIN;
            break;
        }
    }
    // 다음 상태 동작 수행
    ...
}
```

XM_SendIVector()

임피던스 제어 파라미터(I-Vector)를 전송하여, 로봇 관절이 마치 용수철이나 댐퍼처럼 동작하도록 설정합니다. 사전에 **kp**와 **kd**의 최대값을 KIT H10의 구동기 최대 토크인 **10Nm**와 전체 시스템의 동작을 보면서 **신중히** 튜닝해야 합니다. (XM_SendIVectorKpKdMax()) 구동기 최대 전류는 **14A**이고, 모터드라이버 내부 임피던스 제어 입력 생성시 최대 **10A**에서 **Saturation**을 수행하도록 되어 있습니다.



▲ Figure 2. I-Vector(빨강)와 P-Vector(파랑)를 통한 위치 기반 제어 시뮬레이션 예시

Syntax

```
void XM_SendIVector(SystemNodeID_t nodeId, const IVector_t* iVector);
```

Parameters

- nodeId**: 명령을 전달할 관절.
- iVector**: 임피던스 파라미터(Stiffness **kp**, Damping **kd** 등)가 담긴 **IVector_t** 구조체의 포인터.

Returns 없음.

iVector 구조체 주요 멤버:

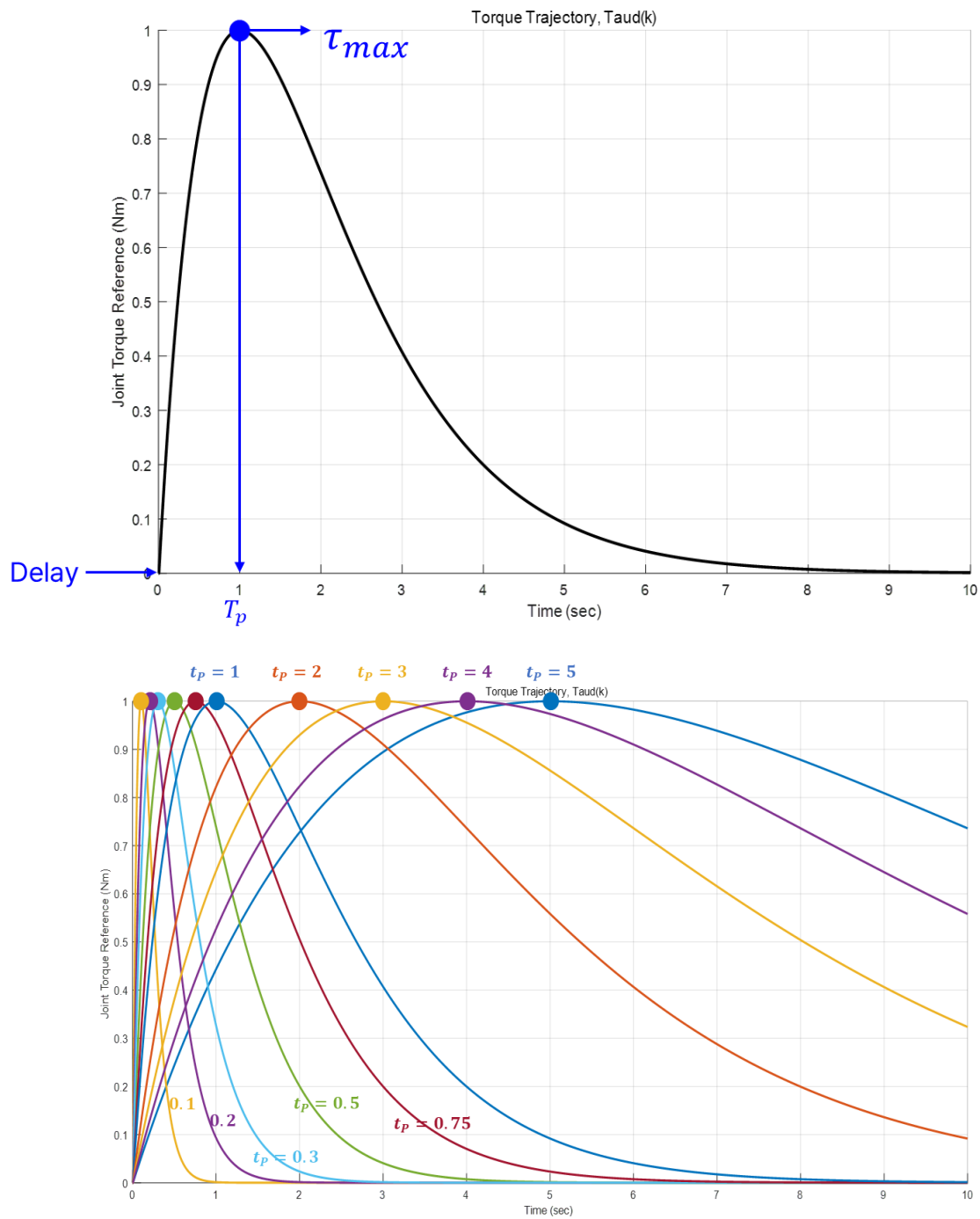
멤버	설명	단위	타입
epsilon	코리더(Corridor)의 절반 폭	degree, scaled by 10	uint8_t
kp	가상 스프링 강도	%	uint8_t
kd	가상 댐퍼 강도	%	uint8_t
lambda	임피던스 비율	ratio, scaled by 100	uint8_t
duration	전환 시간	ms	uint16_t

Example

```
static void EnterStandbyMode(void)
{
    // 임피던스 설정 및 파라미터 해제
    IVector_t stiffImpedance = { .epsilon = 0, .kp = 0, .kd = 0, .lambda = 0,
    .duration = 50 };
    XM_SendIVector(SYS_NODE_ID_RH, &stiffImpedance);
    XM_SendIVector(SYS_NODE_ID_LH, &stiffImpedance);
    XM_ClearPVectorDoneFlag(SYS_NODE_ID_RH);
    XM_ClearPVectorDoneFlag(SYS_NODE_ID_LH);
}
```

XM_SendFVector()

힘 기반 궤적(F-Vector)을 전송하여, 지정된 시간 동안 사전 정의된 토크 궤적을 생성하도록 명령합니다.



▲ Figure 3. -Vector 기반 힘 궤적 생성 예시

Syntax

```
void XM_SendFVector(SystemNodeID_t nodeId, const FVector_t* fVector);
```

Parameters

- **nodeId**: 명령을 전달할 관절.
- **fVector**: 목표 토크(τ_{Max}), 모드($modeIdx$) 등의 정보가 담긴 **FVector_t** 구조체의 포인터.

Returns 없음.

fVector 구조체 주요 멤버:

멤버	설명	단위	타입
<code>modeIdx</code>	토크 프로파일 인덱스	Index (0.1 ~ 10)	<code>uint16_t</code>
<code>tauMax</code>	최대 토크	A, scaled by 100	<code>int16_t</code>
<code>delay</code>	초기 지연 시간	ms	<code>uint16_t</code>
<code>zero</code>	리셋을 위한 더미 데이터	-	<code>uint16_t</code>

Example

(작성 예정)

XM_SendPVectorReset()

현재 실행 중이거나 대기 중인 **P-Vector** 명령을 즉시 초기화(리셋)합니다. 비상 상황이나 사용자의 의도가 바뀌었을 때 현재 동작을 중단시키는 용도로 사용됩니다.

Syntax

```
void XM_SendPVectorReset(SystemNodeID_t nodeId);
```

Parameters

- `nodeId`: P-Vector를 리셋할 관절 (`SYS_NODE_ID_RH` 또는 `SYS_NODE_ID_LH`).

Returns 없음.

Example

```
static void ManageModeTransition(void)
{
    // 현재 모드 값 가져오기
    SuitMode_t currentSuitMode = XM.status.h10.h10Mode;

    switch (s_modeTransitionState) {
        case MODE_TRANSITION_IDLE:
            // 평상시에 모드 변경이 감지되었는지 확인합니다.
            if (currentSuitMode != s_previousSuitMode) {

                // Passive Mode -> Standby Mode 로의 전환
                // P-Vector를 사용하던 Passive Mode를 안전하게 정지시키는 절차를 시작
                // 합니다.
                if (s_previousSuitMode == XM_H10_MODE_ASSIST && currentSuitMode ==
                    XM_H10_MODE_STANDBY) {
                    XM_SendPVectorReset(SYS_NODE_ID_RH); // P-Vector 궤적 생성 취
```


소 명령 전송

```
XM_SendPVectorReset(SYS_NODE_ID_LH);
s_modeTransitionTimer = XM_GetTick(); // reset 지연 타이머 시
```

작

```
s_modeTransitionState = MODE_TRANSITION_STOP_PENDING; // 다음
```

상태로 전환

```
    }
}
break;
// 다음 상태 동작 수행
...
```

XM_ClearPVectorDoneFlag()

XM.status.h10를 통해 isPVectorRHDone 또는 isPVectorLHDone 플래그가 true가 된 것을 확인한 후, 이벤트를 처리했음을 XM10이 알기 위해서는 이 함수를 수동으로 호출해야 합니다. 이 함수를 호출하지 않으면 플래그가 계속 true로 남아 동일한 완료 이벤트가 반복 처리될 수 있습니다. 사용 시 XM.status.h10를 통해 받은 isPVectorRHDone 또는 isPVectorLHDone 플래그가 false로 초기화 됨.

Syntax

```
void XM_ClearPVectorDoneFlag(SystemNodeID_t nodeId);
```

Parameters

- nodeId: 완료 플래그를 클리어할 관절 (SYS_NODE_ID_RH 또는 SYS_NODE_ID_LH).

Example

```
static void EnterStandbyMode(void)
{
    // 임피던스 설정 및 파라미터 해제
    IVector_t stiffImpedance = { .epsilon = 0, .kp = 0, .kd = 0, .lambda = 0,
    .duration = 50 };
    XM_SendIVector(SYS_NODE_ID_RH, &stiffImpedance);
    XM_SendIVector(SYS_NODE_ID_LH, &stiffImpedance);
    XM_ClearPVectorDoneFlag(SYS_NODE_ID_RH);
    XM_ClearPVectorDoneFlag(SYS_NODE_ID_LH);
}
```

XM_SendIVectorKpKdMax()

지정된 관절의 임피던스 제어에서 사용될 최대 Kp(Stiffness)와 Kd(Damping) 값을 설정합니다. 이는 XM_SendIVector에서 백분율(%)로 전달되는 가상 스프링 및 댐퍼 강도의 기준이 됩니다.

Syntax

```
void XM_SendIVectorKpKdMax(SystemNodeID_t nodeId, const float kpMax, const float kdMax);
```

Parameters

- **nodeId**: 파라미터를 설정할 관절 (SYS_NODE_ID_RH 또는 SYS_NODE_ID_LH).
- **kpMax**: XM_SendIVector의 kp 파라미터가 100%일 때 적용될 최대 Kp(가상 스프링 강성) 값입니다.
- **kdMax**: XM_SendIVector의 kd 파라미터가 100%일 때 적용될 최대 Kd(가상 댐퍼 강성) 값입니다.

Example

```
static void InitHoming(void)
{
    static uint32_t homingTimer = 0;
    // --- Homing 상태 머신 ---
    switch (s_homingState) {
        case HOMING_ENTRY:
            XM_SendIVectorKpKdMax(SYS_NODE_ID_RH, 6, 1);
            XM_SendIVectorKpKdMax(SYS_NODE_ID_LH, 6, 1);
            s_homingState = HOMING_SET_IMPEDANCE;
            break;
        // 다음 상태 동작 수행
        ...
    }
}
```

Set... 루틴 및 파라미터 (Routines & Parameters)

Set으로 시작하는 함수들은 KIT H10에 내장된 다양한 제어 보조 루틴을 활성화하거나 관련 파라미터를 실시간으로 조정하는 데 사용됩니다. 이를 통해 사용자는 복잡한 하위 제어 로직을 직접 구현할 필요 없이, 고수준에서 로봇의 동작 특성을 변경할 수 있습니다.

1. 각도 및 각속도 제한 (Angle & Velocity Limit)

로봇의 움직임을 물리적으로 안전한 범위 내로 제한하는 기능입니다.

Syntax

```
// 각도 제한 루틴 활성화/비활성화
void SetDegreeLimitRoutine(SystemNodeID_t nodeId, bool isSet);

// 각도 제한 범위 설정
void SetDegreeLimit(SystemNodeID_t nodeId, float upperLimit, float lowerLimit);

// 각속도 제한 루틴 활성화/비활성화
void SetVelocityLimitRoutine(SystemNodeID_t nodeId, bool isSet);

// 각속도 제한 범위 설정
void SetVelocityLimit(SystemNodeID_t nodeId, float upperLimit, float lowerLimit);
```

Parameters

- **nodeId**: 제어할 관절 (**SYS_NODE_ID_RH** 또는 **SYS_NODE_ID_LH**).
- **isSet**: 해당 루틴을 활성화하려면 **true**, 비활성화하려면 **false**를 전달합니다.
- **upperLimit**: 설정할 가동 범위의 상한값 (단위: degree 또는 deg/s).
- **lowerLimit**: 설정할 가동 범위의 하한값 (단위: degree 또는 deg/s).

Example

```
// 오른쪽 다리의 각도 제한 기능을 활성화하고,
// 가동 범위를 -30도에서 30도 사이로 설정합니다.
XM_SetDegreeLimitRoutine(SYS_NODE_ID_RH, true);
XM_SetDegreeLimit(SYS_NODE_ID_RH, 30.0f, -30.0f);

// 왼쪽 다리의 최대 속도를 100 deg/s로 제한합니다.
XM_SetVelocityLimitRoutine(SYS_NODE_ID_LH, true);
XM_SetVelocityLimit(SYS_NODE_ID_LH, 100.0f, -100.0f);
```

2. 외란 관측기 (Disturbance Observer)

사용자가 가하는 힘이나 예상치 못한 외부 힘(외란)을 추정하고 보상하여, 더 부드럽고 안정적인 움직임을 만들어내는 **KIT H10**에 내장된 고급 제어 루틴입니다. **DOB** 기능을 사용하기 위해서는 **KIT H10**의 구동기가 **DOB**기능에 대한 식별(**System Identification**)이 진행되어 모터드라이버의 **DOB** 식별 정보 기록 여부를 확인해야 합니다.(현재 **KIT H10**은 **DOB** 식별을 진행하지 않았음, 추후 변경 예정) **KIT H10**의 **DOB**에 대해서는 [**angel Robotics-Control Algorithm**](작성 예정)에서 확인할 수 있습니다.

Syntax

```
void XM_SetDOBRoutine(SystemNodeID_t nodeId, bool isSet);
```

Parameters

- **nodeId**: 제어할 관절 (**SYS_NODE_ID_RH** 또는 **SYS_NODE_ID_LH**).
- **isSet**: DOB 루틴을 활성화하려면 **true**, 비활성화하려면 **false**를 전달합니다.

Example

```
XM_SetDOBRoutine(SYS_NODE_ID_RH, true);
```

3. 보상 게인 설정 (Compensation Gain)

KIT H10에 내장된 기본 중력/속도 보상 모드의 강도를 조절합니다. KIT H10의 보상에 대해서는 [angel Robotics-Compensation](작성 예정)에서 확인할 수 있습니다.

Syntax

```
// 일반 보상 게인 설정 (중력 보상 등)
void XM_SetNormalCompGain(SystemNodeID_t nodeId, uint8_t gain);

// 저항 보상 게인 설정 (저항 훈련 모드)
void XM_SetResistiveCompGain(SystemNodeID_t nodeId, float gain);
```

Parameters

- nodeId**: 제어할 관절 (SYS_NODE_ID_RH 또는 SYS_NODE_ID_LH).
- gain**: 설정할 게인 값. 값의 범위와 효과는 각 보상 모드에 따라 다릅니다.

Example

```
// 저항 훈련 모드에서 오른쪽 다리의 저항을 강하게 설정합니다.
float strongResistance = 0.8f;
XM_SetResistiveCompGain(SYS_NODE_ID_RH, strongResistance);
```

XM_SendUserBodyData()

사용자의 신체 정보(몸무게, 키, 분절 길이 등)를 KIT H10로 전송합니다. KIT H10은 이 정보를 바탕으로 실시간 동작 분석을 수행하며 더 정확하고 개인화된 보행 데이터 및 운동 역학 데이터를 계산하여 XM10으로 보내줍니다. KIT H10의 실시간 동작 분석의 자세한 내용은 [GaitAnalysis](작성 예정)에서 확인할 수 있습니다. 사용자가 직접 신체 정보를 측정하여 KIT H10으로 전송해야 합니다.

RxData_t 구조체 중 신체 정보 기반 데이터:

PDO데이터	설명	단위	타입
leftKneeAngle	추정된 왼쪽 무릎 각도	degree	float
rightKneeAngle	추정된 오른쪽 무릎 각도	degree	float
isLeftFootContact	왼쪽 발 접지 여부	-	bool
isRightFootContact	오른쪽 발 접지 여부	-	bool
gaitState	보행 상태	state	uint8_t
gaitCycle	현재 보행 주기	%	uint8_t
forwardVelocity	추정된 전진 속도	m/s	float

...

Syntax

```
void XM_SendUserBodyData(const uint32_t bodyData[8]);
```

Parameters

bodyData 8개의 uint32_t 신체 정보를 담은 배열:

인덱스	설명	단위	타입
0	착용자 몸무게	g	uint32_t
1	착용자 키	mm	uint32_t
2	착용자 오른쪽 허벅지 분절 길이	mm	uint32_t
3	착용자 왼쪽 허벅지 분절 길이	mm	uint32_t
4	착용자 오른쪽 종아리 분절 길이	mm	uint32_t
5	착용자 왼쪽 종아리 분절 길이	mm	uint32_t
6	착용자 오른쪽 발목 분절 길이	mm	uint32_t
7	착용자 왼쪽 종아리 분절 길이	mm	uint32_t

Example

```
// 신체 정보를 입력하고 CM에 전송합니다.
bodyData[0] = 73000; // 73kg
bodyData[1] = 1800;  // 180cm
bodyData[2] = 500;   // 0.5m
bodyData[3] = 495;   // 0.495m
bodyData[4] = 440;   // 0.440m
bodyData[5] = 435;   // 0.435m
bodyData[6] = 60;    // 0.06m
bodyData[7] = 59;    // 0.059m
SendUserBodyData(&bodyData[0]);
```

실시간 제어(Real-time Control)

2ms 제어 루프 내에서 실시간으로 토크를 인가하는 데 사용되는 핵심 함수들입니다. 데이터를 읽는 것은 구조체 접근만으로 가능하지만, 제어 명령(Output)을 내릴 때는 반드시 아래의 Helper 함수들을 사용해야 합니다. 이 함수들은 내부적으로 Dirty Flag를 관리하여 변경된 데이터만 효율적으로 전송하도록 돕습니다.

XM_SetAssistTorque

왼쪽과 오른쪽 다리의 보조 토크를 동시에 설정합니다. 토크를 실제로 전송(Output)하는 것은 Core Process에서 내부적으로 IPO 모델에 따라 처리하고 있습니다. 사용자는 알고리즘 계산을 통해 목표 토크를 해당 함수

를 통해 설정하기만 하면 됩니다.

Syntax

```
void XM_SetAssistTorque(float r, float l);
```

Parameters

- **r**: 오른쪽 고관절 보조 토크 (Unit: Nm)
- **l**: 왼쪽 고관절 보조 토크 (Unit: Nm)

Returns: None

Operating Principle

- 입력된 값을 `XM.command` 구조체에 저장하고, `torque_updated` 플래그를 세팅합니다.
- 실제 전송은 현재 제어 주기의 끝(`_FlushAllOutputs`)에서 이루어집니다.

Example

```
void Active_Loop(void) {  
    // P-Control: 현재 각도에 비례하여 토크 생성  
    float cmd_R = XM.status.h10.rightHipAngle * 0.5f;  
    float cmd_L = XM.status.h10.leftHipAngle * 0.5f;  
  
    // 양쪽 다리에 토크 명령 설정  
    XM_SetAssistTorque(cmd_R, cmd_L);  
}
```

XM_SetAssistTorqueR / XM_SetAssistTorqueL

한쪽 다리의 토크만 개별적으로 설정합니다. 반대쪽 다리의 토크 값은 이전 상태를 유지합니다.

Syntax

```
void XM_SetAssistTorqueR(float r);  
void XM_SetAssistTorqueL(float l);
```

Parameters

- **r / l**: 해당 관절의 보조 토크 (Unit: Nm)

Example

```
// 오른쪽 다리만 5.0Nm로 설정 (왼쪽은 기존 값 유지)  
XM_SetAssistTorqueR(5.0f);
```

유틸리티 (Utilities)

XM_GetTick()

시스템 부팅 후 경과된 시간을 밀리초(ms) 단위로 반환합니다. 특정 동작의 시간을 측정하거나, 일정 시간 동안만 로직을 수행하는 등의 시간 기반 제어에 필수적입니다.

Syntax

```
uint32_t XM_GetTick(void);
```

Returns

- `uint32_t`: 부팅 후 경과된 시간 (ms). 이 값은 약 49.7일마다 0으로 되돌아갑니다(rollover).

Example

```
static void ManageModeTransition(void)
{
    SuitMode_t currentSuitMode = XM.status.h10.h10Mode;

    switch (s_modeTransitionState) {
        case MODE_TRANSITION_IDLE:
            // 평상시에 모드 변경이 감지되었는지 확인합니다.
            if (currentSuitMode != s_previousSuitMode) {

                // Active-Assist Mode -> Standby Mode 로의 전환
                // [CASE 1] Homing 중 P-Vector를 사용하던 AA Mode를 안전하게 정지시
                키는 절차를 시작합니다.
                if (s_previousSuitMode == XM_H10_MODE_ASSIST && currentSuitMode ==
H10_STANDBY_MODE
                    && s_aaGlobalState == AA_STATE_HOMING) {
                    XM_SendPVectorReset(SYS_NODE_ID_RH);    // P-Vector 궤적 생성 취
소 명령 전송

                    XM_SendPVectorReset(SYS_NODE_ID_LH);
                    s_modeTransitionTimer = XM_GetTick();    // reset 지연 타이머 시
작

                    s_modeTransitionState = MODE_TRANSITION_STOP_PENDING; // 다음
상태로 전환
                }
                // 다음 상태 동작 수행
                ...
            }
    }
}
```